AGV2 Kayaba — Compiled Release Guide

How the locked release made by `build_release.sh` works for the customer, and how the developer develops, builds, updates and rolls it back. The AGV runs a compiled program (`/opt/agv2/main.bin`); the Python source stays with the developer. 15 Oct 2026.

1. Customer side (after hand-over)

What the customer gets

- **Power on and it runs.** The controller starts at boot as the `agv-controller` service and restarts itself within 3 s if it crashes.
- **The dashboard opens by itself** full-screen on the AGV's screen (kiosk, if installed) and is reachable at `http://<agv-ip>:5000`.
- **Physical controls:** RESET, branch buttons (start AUTO), CALL START, pusher button DI0, E-stop, mode selector — plus the **USB remote** (WASD / ring, 1–5, Z/X, R, Q, 7–0, E, ENTER/OK).
- **Without a password:** HOME, MANUAL (jog, keyboard, bypass buttons, pusher, AUTO REVERSE), IO lamps, ERRORS, the remote and DI0.

What the customer cannot do

- **See the code.** `/opt/agv2` holds only `main.bin` and libraries — no .py files — and a normal Ubuntu user cannot open the folder.
- **Change the profile** (RFID tags, IO) without the developer's admin (sudo) password, or log in as the service account `agv` (no login shell).
- **Use PARAMS, IO TEST or calibration** without the engineer password (ENGINEER button in the bottom bar, 30-minute session).

> **Note**
> • The kiosk logs in automatically: anyone at the screen gets the desktop after Alt+F4. The compiled code stays protected and sudo still needs your password — never share it. If the customer needs their own Ubuntu login, make it a normal user **without sudo**.

2. Developer: day-to-day development (no rebuild)

While changing and testing, run straight from source — it starts in seconds. Only one copy may run at a time, so stop the release first:

```
sudo systemctl stop agv-controller        # stop the installed release (same ports)
sudo systemctl disable agv-controller     # optional: don't start the release at boot
cd ~/agv2 && git pull
AGV_ID=agv2_kim python3 main.py           # Ctrl+C to stop (exits within a few seconds)
```

- Once per machine for the remote: `sudo usermod -aG input $USER` (log out/in) and `pip3 install evdev`.
- **No internet?** Copy from the Mac over the local network instead of `git pull`:

```
rsync -av --exclude .git --exclude build --exclude _backup --exclude data \
  "<Mac folder>/AGV Kayaba (2 Sensor)/" amrqc-station@<agv-ip>:~/agv2/
```

3. Developer: building the release

```
cd ~/agv2 && git pull          # or rsync from the Mac
./build_release.sh             # compile + install to /opt/agv2 + enable and start the service
```

- **Internet:** only needed when tools or libraries are missing (first build, or after a new library such as evdev). Otherwise the script prints “Already installed — skipping apt/pip” and builds offline.

- Installs to `/opt/agv2` (root-owned, readable only by agv), adds agv to groups `dialout` and `input`, writes the systemd unit (with `TimeoutStopSec=15`) and the restart-button sudoers rule, starts the service and checks the dashboard answers.
- **Kept across rebuilds:** the engineer password and the saved PARAMS tuning (`/opt/agv2/data/`). The previous release is kept as `/opt/agv2.old-<date>`.
- Nuitka is pinned below 4.3 and GTK is left out (4.3 crashes on Ubuntu 22.04). `--build-only` compiles without installing.

Dashboard on the AGV screen at power-on (once)

```
sudo snap install chromium           # recommended (cleanest kiosk)
bash kiosk/install_kiosk.sh          # auto-login + autostart + no screen lock
sudo reboot                          # undo: bash kiosk/install_kiosk.sh --remove
```

Hand-over

Test everything on the real AGV — AUTO, branch 2 / 1C drop, reverse + CC hitch, pusher, lidar, remote, all web pages — then remove the source:

```
rm -rf ~/agv2                          # source + .git (your copy stays on GitHub)
gh auth logout; rm -f ~/.git-credentials
sudo systemctl disable --now ssh       # unless you want remote access
```

4. Everyday commands on the AGV

Task	Command
Is it running?	`sudo systemctl status agv-controller`
Live log	`sudo journalctl -u agv-controller -f`
Restart / stop	`sudo systemctl restart agv-controller · sudo systemctl stop agv-controller`
Change a tag on site	`sudo nano /opt/agv2/profiles/agv2_kim.json (then restart)`
Reset saved tuning	`PARAMS → RESET TO PROFILE (or delete /opt/agv2/data/tuning_agv2_kim.json)`
Engineer password	`sudo AGV_ID=agv2_kim /opt/agv2/main.bin --set-engineer-password`
RFID reader settings	`python3 debugging/rfid_settings.py read (controller stopped, source folder)`
Which keyboards are seen	`python3 debugging/list_keypads.py`

Rolling back a bad build

```
sudo systemctl stop agv-controller
sudo mv /opt/agv2 /opt/agv2.bad
sudo mv /opt/agv2.old-<date-time> /opt/agv2
sudo systemctl start agv-controller
```

5. Watch out for

Note
• **Push before you pull.** “Already up to date” on the AGV means the Mac commits were not pushed in GitHub Desktop.
• **Profile edits made on the AGV are replaced by the next build.** Copy on-site changes back into `agv2_kim.json` in the repo first. Saved PARAMS tuning wins over the profile for the tuned values — use RESET TO PROFILE if a profile edit seems ignored.
• **Always build on the AGV itself** (or an identical Ubuntu + CPU). Keep the source safe: private GitHub repo plus a backup on the Mac.
• **Stronger protection:** BIOS/UEFI password, USB boot disabled, LUKS disk encryption with TPM unlock. State in the sale agreement that the software is licensed for this AGV only (no copying or reverse engineering).